Audio Dataset Quality Assurance (QA) Pipeline Report

Sprint 1 Dataset Validation and Reporting Pipeline

---

1. Introduction

This project implements a reusable Dataset Quality Assurance (QA) pipeline designed to inspect, validate, and report the quality of an existing bioacoustic audio dataset.

The purpose of the pipeline is to perform automated quality checks before machine learning development, ensuring that the dataset structure, metadata, and audio characteristics are understood before model training.

Bioacoustic datasets commonly contain issues such as:

- inconsistent directory structures,
- missing source or species labels,
- unreadable audio recordings,
- unsupported file formats,
- unexpected recording durations,
- inconsistent metadata,
- low-quality audio recordings.

The developed pipeline addresses these problems through automated validation while maintaining the original dataset unchanged.

The pipeline follows a non-destructive quality assessment approach:

- files are not deleted,
- files are not renamed,
- audio files are not permanently modified,
- detected issues are recorded for review.

This follows the Sprint 1 requirement:

Avoid deleting records automatically during Sprint 1.

The purpose of the pipeline is therefore not to automatically clean or modify the dataset, but to produce a reliable dataset-quality report that allows researchers to make informed decisions before further processing.

Scope of Quality Assessment

The implemented pipeline focuses on validating the supplied audio recordings and their associated directory structure. Metadata validation (e.g., recording dates, geographic coordinates, taxonomic information or annotation files) is outside the scope.

---

2. Dataset Source and Structure Assumptions

2.1 Dataset Availability During Sprint 1

The original project handover referenced multiple potential biodiversity data sources, including:

- Google Cloud Platform (GCP) datasets,
- iNaturalist,
- Atlas of Living Australia (ALA).

During Sprint 1 implementation, it was identified that the iNaturalist and ALA datasets referenced in the handover were potential future data sources rather than available dataset files stored within the project environment.

No complete iNaturalist or ALA audio datasets were available for direct pipeline execution.

Therefore, the current notebook implementation was developed and demonstrated using the available GCP dataset.

The active validation input consists of:

- audio files available from the GCP dataset,
- existing folder structures,
- available source and species information,

The pipeline does not claim to validate iNaturalist or ALA datasets during Sprint 1 because those datasets were not available as actual input files.

2.2 Source-Aware Dataset Design

Although only the GCP dataset is currently used for execution, the pipeline was designed to support multiple future data sources.

The dataset structure assumption is:

datasets/

source/

species/

audio_file.wav

Example:

datasets/

gcp/

eastern_rosella/

recording_001.wav

The pipeline extracts:

Field	Example
source	gcp
species	eastern_rosella

This allows future datasets from different providers to be included without changing the validation workflow.

For example, future datasets could follow:

datasets/

gcp/

species_a/

audio.wav

inaturalist/

species_b/

audio.wav

ala/

species_c/

audio.wav

The pipeline would then be able to generate source-level and source-species-level statistics.

However, because only GCP data was available during Sprint 1, the generated reports currently contain only the GCP source records.

2.3 Reason for Using Available Dataset Sources

Using the available GCP dataset ensures that:

1. The pipeline is tested on real input data.
2. Validation results represent the actual available dataset.
3. Outputs are reproducible.
4. No assumptions are introduced from unavailable external datasets.

The purpose of Sprint 1 is dataset quality assessment rather than comparison between biodiversity data providers.

Therefore, the pipeline focuses on:

- file discovery,
- source and species organisation,
- label consistency,
- audio readability,
- file format validation,
- duration checks,
- acoustic quality indicators.

Future datasets can be added by following the same source/species folder structure.

---

3. Pipeline Design Philosophy

The pipeline follows four main principles.

3.1 Detect Rather Than Modify

The system identifies dataset issues but does not automatically fix them.

Examples:

Issue	Pipeline Action
Missing species label	Flag as unknown
Missing source folder	Flag as invalid structure

Issue	Pipeline Action
Corrupted audio	Mark as unreadable
Short recording	Flag duration issue
Unsupported format	Mark as unsupported

This prevents irreversible decisions during early dataset assessment.

3.2 Preserve Original Information

The pipeline preserves:

- original filenames,
- original file paths,
- original sampling rates,
- original channel information,
- original audio recordings.

The pipeline only creates additional metadata fields and reports.

This ensures future preprocessing decisions remain under researcher control.

3.3 Generate Audit Reports

All validation outputs are stored as structured CSV reports.

These reports provide a traceable record of:

- files inspected,
- validation checks performed,
- detected problems,
- affected recordings.

The generated reports allow researchers to review dataset quality before model development.

3.4 Reproducible Execution

The notebook workflow is designed so that another user can reproduce the analysis using:

- the same dataset,
- the same notebook,
- the same configuration settings.

The dataset location and names are configurable rather than permanently fixed.

A standalone .py version can be created in the future for reusable execution across datasets while maintaining the same validation logic.

The notebook remains the primary Sprint 1 demonstration and documentation environment.

---

4. Dataset Configuration and Reproducibility Design

4.1 Portable Dataset Configuration

The pipeline avoids machine-specific dataset paths.

Instead of using:

DATASET_PATH = "/Users/example/Desktop/project/data"

the notebook uses configurable project-relative paths:

PROJECT_ROOT = Path.cwd()

DATASET_ZIP_NAME = "datasets.zip"

DATASET_FOLDER_NAME = "datasets"

The dataset names can be changed depending on the provided files.

Example:

DATASET_ZIP_NAME = "bird_audio.zip"

DATASET_FOLDER_NAME = "bird_audio"

This allows the notebook to be adapted to different datasets without modifying the validation functions.

4.2 Dataset ZIP Handling

The notebook supports compressed dataset input.

Example:

project/

notebook.ipynb

datasets.zip

reports/

The pipeline checks whether the dataset folder exists.

If not, the ZIP archive can be extracted:

```
with zipfile.ZipFile(
    ZIP_PATH,
    "r"
) as zip_ref:

    zip_ref.extractall(
        PROJECT_ROOT
    )
```

This improves portability because large datasets can be transferred as compressed files while maintaining the expected folder structure.

---

5. Environment Verification

To improve reproducibility, the notebook records the computational environment used during execution.

The purpose is to record:

- Python version,
- operating system information,
- installed package versions.

This is important because differences in software versions may affect:

- audio decoding,
- numerical calculations,
- feature extraction.

6. Dataset Discovery

6.1 DatasetScanner Class

The DatasetScanner class performs the initial inspection of the supplied dataset. It recursively scans the dataset directory and creates a manifest containing one record for every discovered file.

For each file, the scanner records:

- filename,
- absolute file path,
- relative file path,
- file extension,
- file size.

This manifest forms the basis for all subsequent validation, reporting and quality assessment performed by the pipeline.

---

7. Dataset Structure Validation

7.1 Folder Structure Validation

The pipeline validates whether the dataset follows a consistent folder structure.

The expected directory hierarchy is:

datasets/

source/

species/

audio_file

For example,

datasets/

gcp/

eastern_rosella/

recording.wav

From this structure, the pipeline can identify both the recording source and the corresponding species label.

The validation checks for:

- missing source folders,
- missing species folders,
- inconsistent folder depth.

Rather than automatically modifying the dataset, any inconsistencies are recorded in the dataset manifest and reported for later review.

---

8. Source and Species Extraction

The notebook extracts both the recording source and species label directly from the folder structure.

For the expected structure:

datasets/

gcp/

eastern_rosella/

recording.wav

the extracted values are:

source = gcp

species = eastern_rosella

If either folder cannot be identified, the corresponding value is recorded as **unknown**.

This approach avoids relying on filenames or external metadata while remaining applicable to datasets containing recordings from multiple sources.

---

9. Label Normalisation

Species names often contain inconsistent capitalisation, spaces or special characters.

To ensure consistent class labels, the pipeline normalises all extracted species names by:

- converting text to lowercase,
- replacing non-alphanumeric characters with underscores,
- removing repeated underscores,
- trimming leading and trailing separators.

For example,

Red-browed Pardalote

becomes

red_browed_pardalote

Consistent labels prevent duplicate classes caused by formatting differences.

---

10. Dataset Structure Report

The notebook generates a reusable structure validation report summarising common dataset issues.

The report includes:

- empty files,
- missing species labels,
- unsupported file extensions.

In addition to the summary report, unsupported file extensions are also recorded within the dataset manifest using the format_status field, allowing problematic files to be identified individually.

This validation is non-destructive and does not modify or remove any dataset records.

---

11. Audio Validation

The validate_audio() function performs automated validation of each audio recording.

The validation records:

- readability,
- recording duration,
- original sampling rate,
- number of audio channels.

Audio Validation Variables and Their Importance

The validate_audio() function generates a standard validation record for every audio file. These variables provide information about whether the file can be processed, its basic audio properties, and any detected quality issues.

The output structure is:

result = {

"readable": False,

"duration": None,

"sample_rate": None,

"channels": None,

"issue": ""

}

Each variable has a specific role in dataset quality assessment.

Readable

"readable": False

The readable variable indicates whether the audio file can be successfully opened and decoded by the audio processing library.

During validation:

audio, sr = librosa.load(

path,

sr=None,

mono=False

)

If the file loads successfully:

result["readable"] = True

If an exception occurs during loading:

except Exception as e:

result["issue"] = "unreadable"

The recording is marked as **unreadable**.

Importance

This check identifies files that cannot enter later processing stages. Unreadable files may occur because of:

- corrupted audio files,
- incomplete downloads,
- unsupported encoding formats,
- damaged file headers,
- invalid audio containers.

The pipeline records these issues but does not delete the files because the cause may require manual investigation.

Duration

"duration": None

The duration variable stores the length of each recording in seconds.

It is calculated using:

duration = librosa.get_duration(

```
    y=audio,
    sr=sr
)
```

The value is rounded:

```
result["duration"] = round(
    duration,
    3
)
```

Importance

Recording duration is important because extremely short or long recordings can affect machine learning workflows. Many bioacoustic classification systems use recordings of a consistent duration as model inputs, evaluation samples, or training segments. However, the appropriate duration depends on the target species, model architecture, and research objectives.

In this pipeline, the configured limits (MIN_DURATION = 1.0 second and MAX_DURATION = 10.0 seconds) are used as quality indicators to identify unusually short or long recordings. These values provide a consistent dataset check but are not universal biological requirements. Recordings outside these limits are flagged for review rather than automatically removed.

The pipeline uses:

```
MIN_DURATION = 1.0


MAX_DURATION = 10.0
```

The validation rules are:

Duration	**Flag**
Less than 1 second	too_short
Between 1 and 10 seconds	No issue
Greater than 10 seconds	too_long

These limits are quality indicators rather than automatic rejection rules. A short recording may still contain useful information, while a long recording may require segmentation depending on the future modelling approach.

Sample Rate

"sample_rate": None

The sample_rate variable records the original sampling rate of the supplied audio file.

The pipeline intentionally uses:

librosa.load(

path,

sr=None,

mono=False

)

The parameter:

sr=None

prevents automatic resampling and preserves the original recording metadata.

Importance

Sampling rate determines the frequency range that can be represented in a recording. However, there is no universal sampling rate that is valid for all bioacoustic applications.

The appropriate sampling rate depends on:

- target species,
- frequency range of animal sounds,
- recording equipment,
- downstream model requirements.

For example, bird recordings, ultrasonic bat recordings, and underwater acoustic recordings may require very different sampling rates.

Therefore, the pipeline records sampling rate information but does not automatically classify values as valid or invalid. This avoids making modelling assumptions during dataset quality assessment.

Channels

"channels": None

The channels variable records whether the audio is mono or multi-channel.

The pipeline checks:

```
if audio.ndim == 1:

    channels = 1

else:

    channels = audio.shape[0]
```

A single-dimensional waveform is treated as mono, while multi-dimensional audio is treated as multi-channel.

Importance

Channel information can provide important context about the recording.

For example:

- Mono recordings contain a single audio signal.
- Stereo recordings may contain spatial information.
- Multi-channel field recordings may contain information from multiple microphones.

The pipeline preserves this information instead of converting all recordings to mono because channel information may be useful for future analysis.

Issue

"issue": ""

The issue variable stores detected validation problems.

Possible values include:

Issue	Meaning
unreadable	Audio file could not be opened or decoded
too_short	Recording duration is below 1 second
too_long	Recording duration exceeds 10 seconds
empty	No detected issue

Importance

Instead of modifying the dataset, the pipeline records detected problems so when modeling team can review affected files.

This follows the Sprint 1 requirement:

Detect and report dataset problems without automatically deleting or altering records.

The notebook intentionally preserves the original sampling rate (sr=None) so that the recorded metadata accurately reflects the supplied dataset rather than a resampled version.

All validation results are stored within the dataset manifest.

Sampling Rate Assessment

The pipeline records the original sampling rate for every audio recording but intentionally does not classify any sampling rate as invalid.

Sampling rate suitability depends on the intended application, target species, recording hardware, and future machine-learning requirements. Different bioacoustic applications may require different sampling rates; for example, high-frequency species may require higher sampling rates, while lower-frequency recordings may use lower sampling rates specially in this project with different species categories.

Therefore, the pipeline preserves the original sampling rate using:

```
librosa.load(
    path,
    sr=None
)
```

This ensures that the dataset manifest reflects the supplied recording metadata rather than a resampled version.

The pipeline does not apply fixed thresholds because a sampling rate that appears unusual may still be scientifically valid depending on the recording purpose. Instead, sampling-rate suitability should be evaluated during later modelling or metadata-quality analysis stages.

Values such as unusual or non-standard sampling rates (for example, unexpected fractional values) may indicate metadata or file-export issues rather than an actual recording problem. These cases should be investigated using file metadata and recording information rather than automatically classified as invalid by the dataset QA pipeline.

12. Dataset Statistics

To satisfy the Sprint 1 requirement to report dataset composition, the notebook generates three summary reports.

12.1 Species Counts

The species count report summarises the number of recordings available for each species.

This allows researchers to identify class imbalance before machine learning development.

12.2 Source Counts

The notebook also reports the number of recordings contributed by each data source.

The current demonstration dataset contains recordings only from the Google Cloud Platform (GCP). However, the implementation supports multiple data sources without modification, allowing additional sources such as ALA or iNaturalist to be incorporated in future datasets.

12.3 Source-by-Species Counts

A combined report summarises the number of recordings available for each species within each recording source.

This report provides a more detailed view of dataset composition and satisfies the Sprint 1 requirement to report sample counts by both source and species.

12.4 Invalid Records

This step identifies all invalid records within the manifest DataFrame by applying a series of validation checks. Any record that fails at least one validation criterion is extracted into a separate DataFrame called invalid_records. This allows invalid records to be isolated for reporting, review, or further investigation, while ensuring that only valid records continue through the subsequent stages.

First, it identifies records where the readable column is False, indicating that the data could not be successfully read or interpreted. Second, it checks whether the value in the format_status column is different from "OK", which indicates that the record failed the required format validation. Finally, it checks the issue column to determine whether it contains a valid issue description by ensuring that the value is neither missing (NaN) nor an empty string.

These three conditions are combined using the logical OR (|) operator, meaning that a record is included in invalid_records if any one of the conditions evaluates to True

---

13. Acoustic Complexity Analysis

As an additional quality assessment step, the notebook calculates several acoustic indicators for each recording.

These include:

- silence ratio,
- mean signal energy,
- energy variance,
- spectral centroid.

For consistency, recordings are temporarily resampled to **16 kHz** during feature extraction only. This does not modify the original recordings or the original sampling rates stored in the dataset manifest.

Based on these measurements, each recording is assigned one of the following quality indicators:

Quality Flag	Description
GOOD	Recording appears suitable for analysis
HIGH_SILENCE	Large proportion of silence detected
LOW_SIGNAL	Weak overall signal strength
HIGH_VARIABILITY	Large variation in signal energy
ERROR	Audio processing failed

If feature extraction cannot be completed, the corresponding error message is recorded while allowing the remainder of the pipeline to continue.

---

14. Report Generation

The notebook exports several reusable CSV reports summarising the results of the quality assessment.

Generated reports include:

Report	Description
dataset_manifest.csv	Complete inventory of files and validation results
structure_report.csv	Summary of structural validation checks
species_counts.csv	Number of recordings per species
source_counts.csv	Number of recordings per source
source_species_counts.csv	Number of recordings for each source-species combination
acoustic_complexity_report.csv	Acoustic quality indicators for each recording

Together, these reports provide a reusable summary of dataset quality while preserving the original dataset unchanged.

---

15. Python Script for Reusability

Although the notebook demonstrates the complete workflow interactively, the same pipeline can also be exported as a standalone Python (.py) script.

Providing the pipeline as a Python script improves reproducibility by allowing the same quality assurance process to be executed outside the notebook environment, such as from a command line or as part of an automated workflow. Users only need to provide the expected dataset folder (or dataset ZIP archive) before executing the script.

---

16. Alignment with Sprint 1 Requirements

The completed notebook satisfies the Sprint 1 requirements as follows.

Sprint 1 Requirement	Notebook Implementation
Review existing dataset structures	Dataset discovery and folder validation
Create automated checks for missing files	Empty file detection
Create automated checks for unreadable audio	Audio validation
Create automated checks for missing labels	Source and species extraction
Create automated checks for inconsistent paths	Folder structure validation
Report sample counts by source and species	Species counts, source counts and source-by-species counts
Identify invalid durations	Duration validation
Identify unsupported formats	File format validation
Record sampling rates	Original sampling rates stored in the manifest
Produce a reusable dataset quality report	CSV report generation
Avoid deleting records automatically	Entire pipeline follows a non-destructive approach

This notebook therefore provides a reusable, portable and non-destructive dataset quality assurance pipeline that establishes a validated foundation for subsequent machine learning development while fully satisfying the Sprint 1 objectives.

Important

The pipeline generates a flagged-record list containing files that require further review, such as unreadable audio, unsupported formats, missing labels, or recordings outside configured duration limits.

These flags do not automatically classify recordings as invalid. A flagged recording may still be useful depending on the research objective, target species, and future preprocessing requirements. For example, a short recording may contain a valid species call, and an alternative audio format may still be usable.

During Sprint 1 demonstration, validation was performed on an available sample subset of 39 audio files form the project data set , 5 more from an outside source (xero-canto/not the same species from the project data set) and also invalid formats 2 images. Therefore, the generated flagged-record output represents the demonstrated subset rather than the complete 8000+ file dataset.

When executed on the complete dataset directory or any other data set , the same pipeline can automatically generate a complete flagged-record report by applying the existing validation checks to all available recordings.